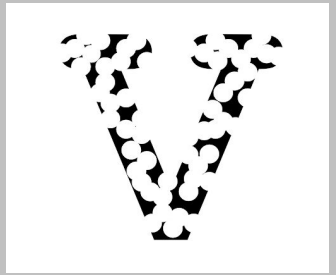
VACUUMMS

A Variational Approach to Void Space Connectivity and Enhanced Accessibility in Amorphous Materials Analysis

Frank T Willmore, Ph.D.
Independent Researcher
Free Volume Institute
vacuumms.org



Why Free Volume?

- No space to move → no motion, nothing happens
- compressibility is possible because of free space
- non-equilibrium phenomena, e.g. permeability depend on free volume
- materials aging
- depletion forces

VACUUMMS challenges

- no explicit conservation laws for voids
- tracking the evolution of voids and channels
- lack of tools

Major components in VACUUMMS

- Cavity Energetic Sizing Algorithm (CESA)
- Free Volume Indexing
- Variational path calculation

VACUUMMS takes the approach of inserting additional 'ghost' atoms.

Minor components (in CLI) include cavity overlaps, rattle volumes, two-point correlations, Voronoi utilities and visualization tools.

VACUUMMS Software Releases

original unversioned release in 2012 (*per* DOI)

version 1.0.x:

- first release of modern VACUUMMS
- introduced semantic versioning
- CMake build system generator

version 1.1.x:

- spack package
- ARM64 support
- many refinements and much debugging of original CLI codes

version 1.2.x introduced:

- limited C++ interface/API
- Variational module

VACUUMMS 1.3.0 Latest

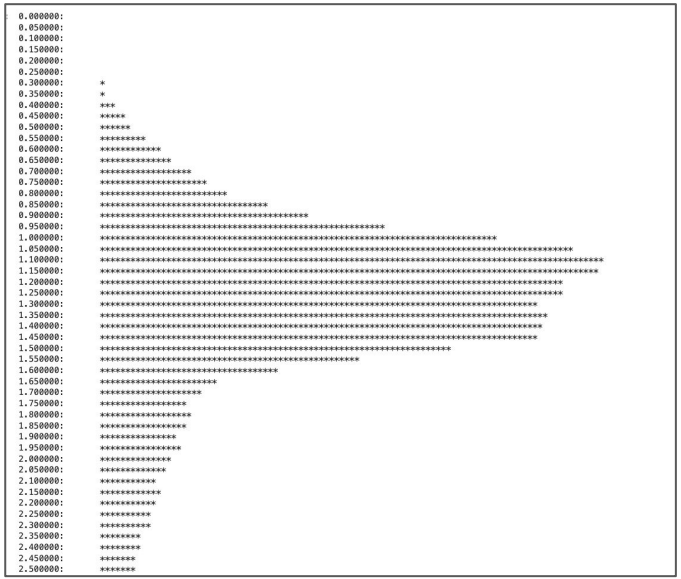
frankwillmore released this 16 hours ago 1.3.0 f5e8a2d

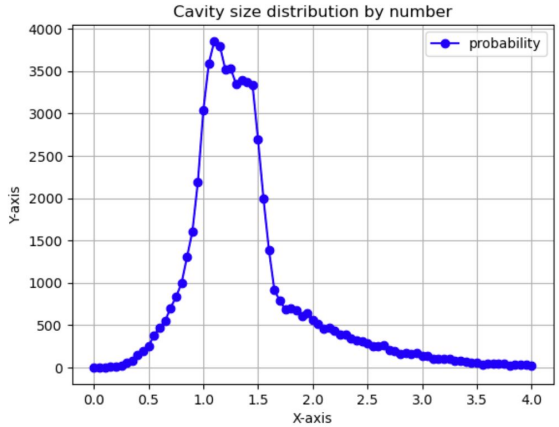
- Full-fledged release of C++/Python/Jupyter notebook support for major codes
- Re-worked DDX algorithm (not yet in PDDX) in C++/python
- Variational module integrated into default build system
- Python/JuPyter Examples for simple FCC model and Variational calculation
- Python/JuPyter Examples of PS (polystyrene) and PMP (polymethylpentene)
- Deprecates but still supports original CLI (command line interface) applications and utilities
- LAMMPS reader implemented and available in C++/python
- Introduces Conda recipe and package for python/JuPyter integration

Cavity Energetic Sizing Algorithm (CESA) Cavities†

- A ghost particle is inserted at a randomly selected point into a simulated molecular sample.
- The particle then slides *via* gradient descent to the point of lowest insertion energy.
- The particle is scaled so that its attractive and repulsive components are equal in magnitude.

CSD for polystyrene sample:





†Liquid Structure *via* Cavity Size Distributions,
Pieter J. in ‘t Veld, Matthew T. Stone, Thomas M. Truskett, Isaac C. Sanchez
https://pubs.acs.org/doi/10.1021/jp001934c

Free Volume Indexing

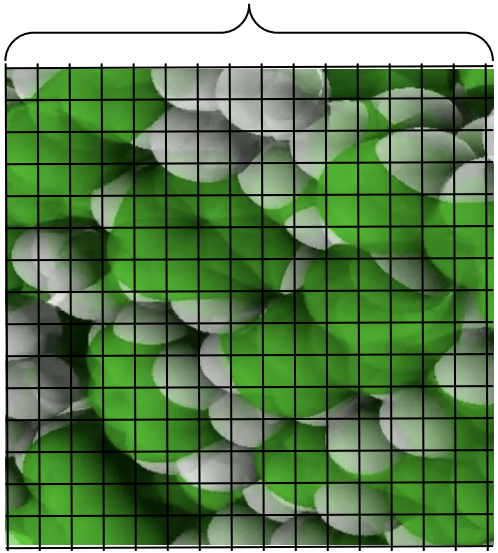
A ghost particle is inserted at regularly spaced intervals into a simulated molecular sample.

The interaction energy of the particle is calculated, with emphasis on repulsive component.

Applying the exponential function to the repulsive component (domain values of [0, inf) generates the free volume index function, whose values range from 0 (completely occupied) to 1 (completely empty).

This free volume index function is readily mapped to the color channels of a TIFF-formatted file, allowing for visual inspection and visualization of the negative space without atoms.

3-D domain decomposition



Chemical potential:

$$\mu_i = \left(\frac{\partial U}{\partial N_i}\right)_{S,V,N_{j\neq i}} \cdot \quad \mu_i = -k_B T \ln\left(\frac{\mathbf{B}_i}{\rho_i \lambda^3}\right)$$

Wisdom insertion parameter:

$$\mathbf{B}_i = \frac{\rho_i}{a_i} = \left\langle \exp\left(-\frac{\psi_i}{k_B T}\right)\right\rangle$$

Free volume index:

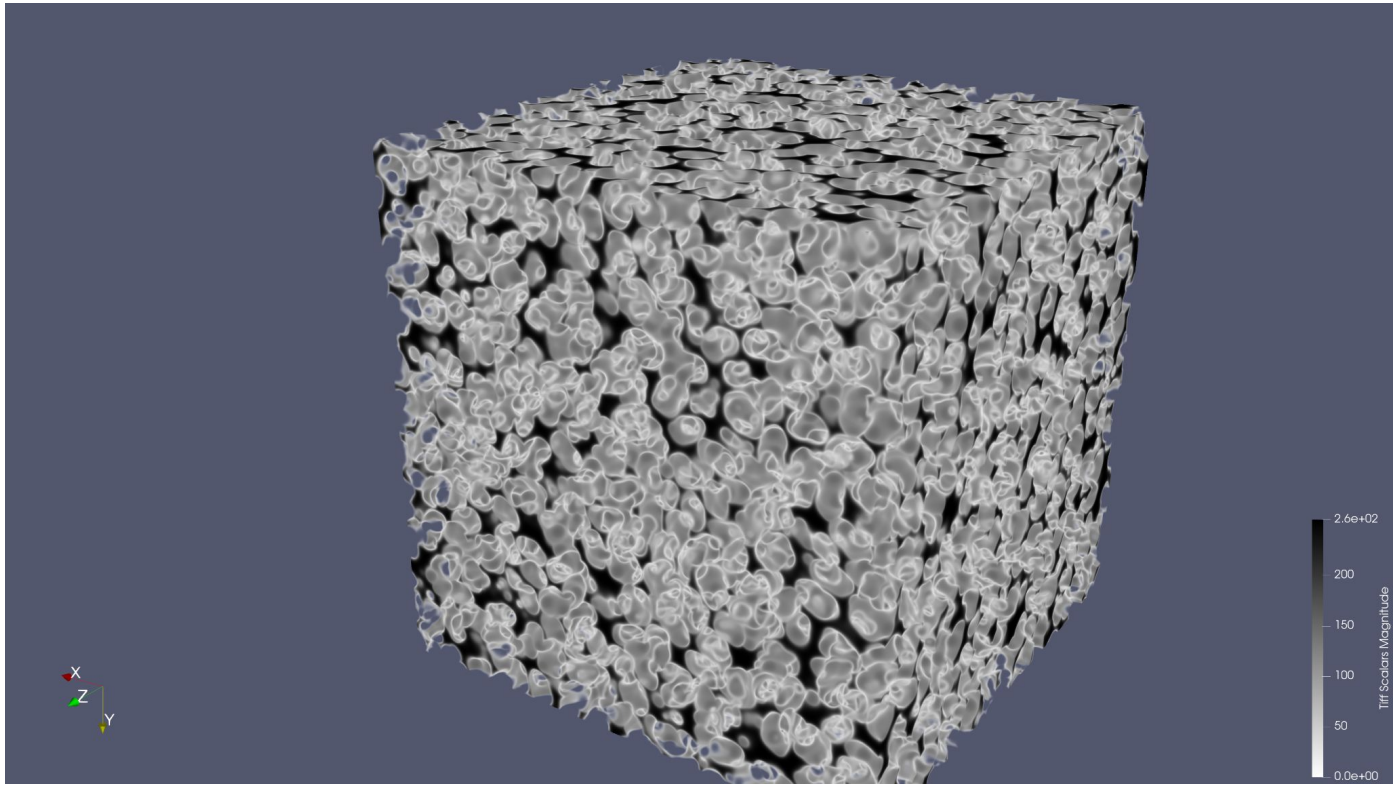
$$\phi_i(x,y,z) = e^{-\beta \gamma_i}$$

Free Volume Indexing Example

A simulated sample of polystyrene (~60k atoms) in a periodic cell.

The free volume index was calculated at 1024x1024x1000 evenly spaced grid points *via* GPU.

The data was used to generate a TIFF image file, which was then imported into the Paraview visualization software.



Adding a Variational Approach to Free Volume Connectivity

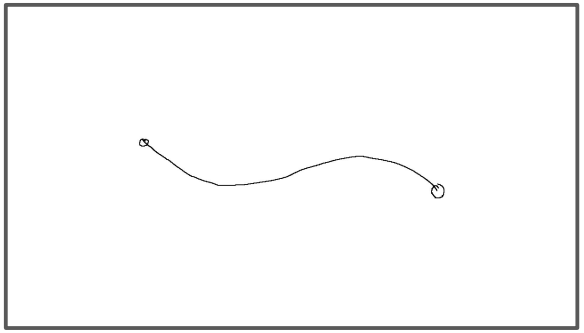
- Complimentary to Molecular Mechanics / Statistical Mechanics approaches
- possible entrypoint to a transition state perspective
- intuition

The question: *What happens when we form a trajectory between two known, existing voids in a simulated sample of material, and then minimize the insertion energy at every point along the trajectory, using only perturbations perpendicular to the trajectory?*

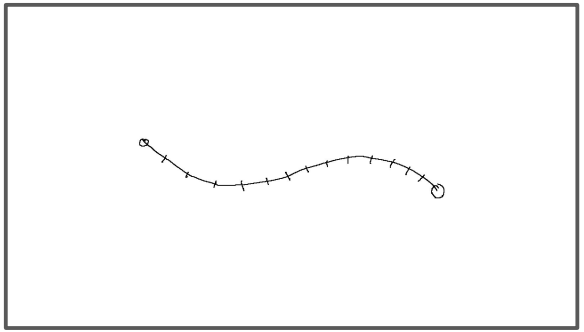
- How can we characterize these paths in a meaningful, insight-generating way?
- How do they evolve over time?
- What is the relationship between these paths and the motion of a diffusing penetrant?
- How does the insertion energy vary along theses paths, and can the maxima be used as inputs for transition state models?
- What additional questions do they allow us to ask, or to potentially answer, of a sample?

Variational calculation: iteration step

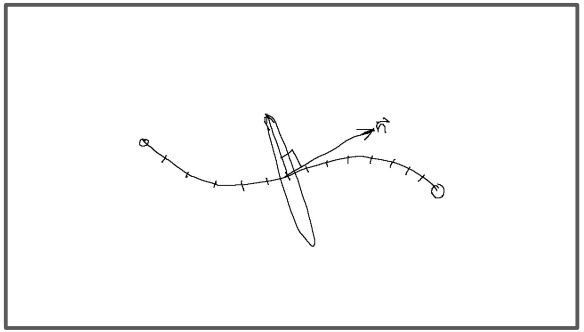
A pair of end points (e.g. cavity centers) is selected and an initial trajectory (default is a straight line) is drawn between them.



This initial trajectory is then partitioned into equally spaced 'variational points' for the numerical calculation.



For each of the intermediate variational points, a vector tangent to the trajectory is used to define the plane perpendicular to the trajectory.



Calculation of gradient in perpendicular plane

Each variational point *p* is neighbored by a point before and a point after (one of being the start or end point of the variational curve itself for the first and last variational points, respectively). These two neighboring points are used to define an axis perpendicular (within numerical approximation) to the variational curve at the variational point of interest. Let these points be represented as p_fore and p_aft and the unit vector n tangent to the variational curve as:

n = (p_aft - p_fore) / | p_aft - p_fore |

The direction of these derivatives is derived by rotating the (x, y, z) coordinate system such that the z-axis points in the direction n of the variational path at the variational point being evaluated. The parameters of this rotation are determined from mapping k, the unit vector in the z-direction to n.

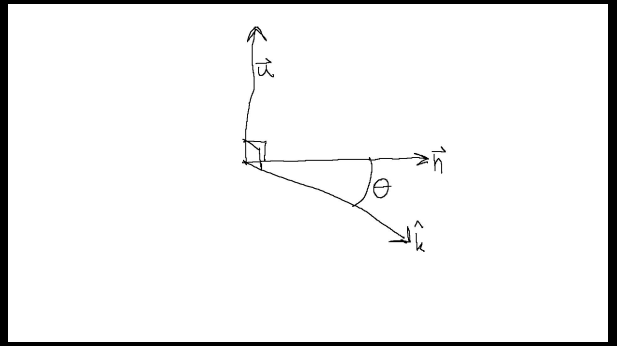
This mapping of the original coordinate system is done by finding a rotation axis u and an angle of rotation *theta* resulting in the unit vector k mapping to the unit vector n. The same rotation maps the i and j unit vectors (pointing along the original x and y axes) to orthogonal vectors directional_x and directional_y, forming a basis set that spans the plane perpendicular to the variational path at the point of interest.

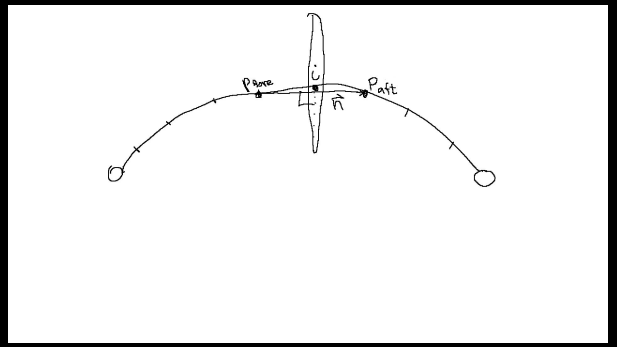
The axis of rotation u can be taken from the cross product:

$$k \times n = \begin{vmatrix} i & j & k \\ 0 & 0 & 1 \\ n_x & n_y & n_z \end{vmatrix}$$

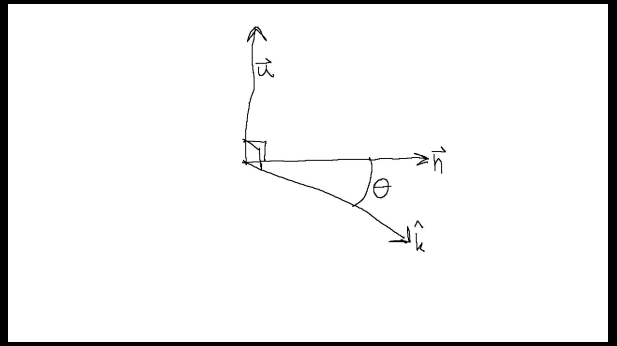
where u is a unit vector pointing in the direction of the cross product:

u = k X n / |k X n|





Calculation of gradient in perpendicular plane (continued)



The value of *theta* can most efficiently be extracted from the dot product:

$k * n = |k||n|\cos(theta)$

$theta = \arccos(k * n)$

This gives values of theta ranging from -PI/2 to PI/2. For rotations larger/smaller than this range, the direction of rotation emerges as an inverted sign in the value of the cross product, i.e. a rotation angle greater or less than values in this range is described by a rotation about an axis pointing in the opposite direction.

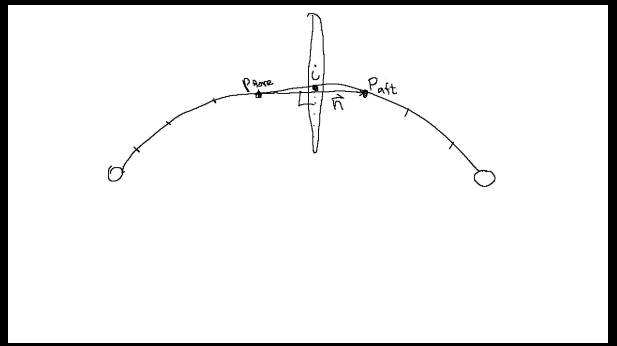
The angle *theta* and the axis u define the following quaternion-based mapping of the rotation:

$q = cos(theta/2) + u\ sin(theta/2)$

$v' = qvq^*$

where v is the original vector and v' is the vector in the rotated coordinate system, and q* is the quaternion conjugate of q. Thus, k rotates to n and by definition, i and j rotate to directional_x and directional_y respectively. These directionals are used to evaluate the directional gradient in the potential field E and thus generate perturbations:

$p_{n+1} = p_n + alpha * grad\ E(p_n)$

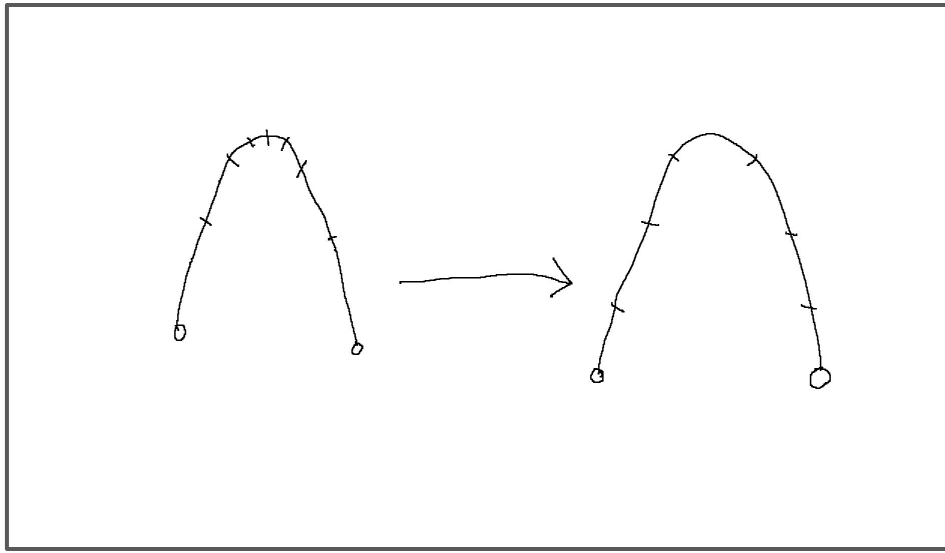


Variational calculation update step

The two-dimensional gradient of the insertion energy constrained to this perpendicular plane is used to perform gradient descent of the trajectory using an adaptive (ADAM) algorithm. The perturbation at each point is calculated separately, with the entire curve being updated simultaneously to conclude a single iteration of the algorithm.

Upon applying the calculated perturbations to the positions of the set of variational points, the variational curve is stretched unevenly, and undergoes a 'rebalancing' step in which the points are re-spaced equidistantly along the trajectory.

Iteration continues until convergence is satisfactorily reached, or perturbation size falls below a threshold, e.g. machine epsilon.



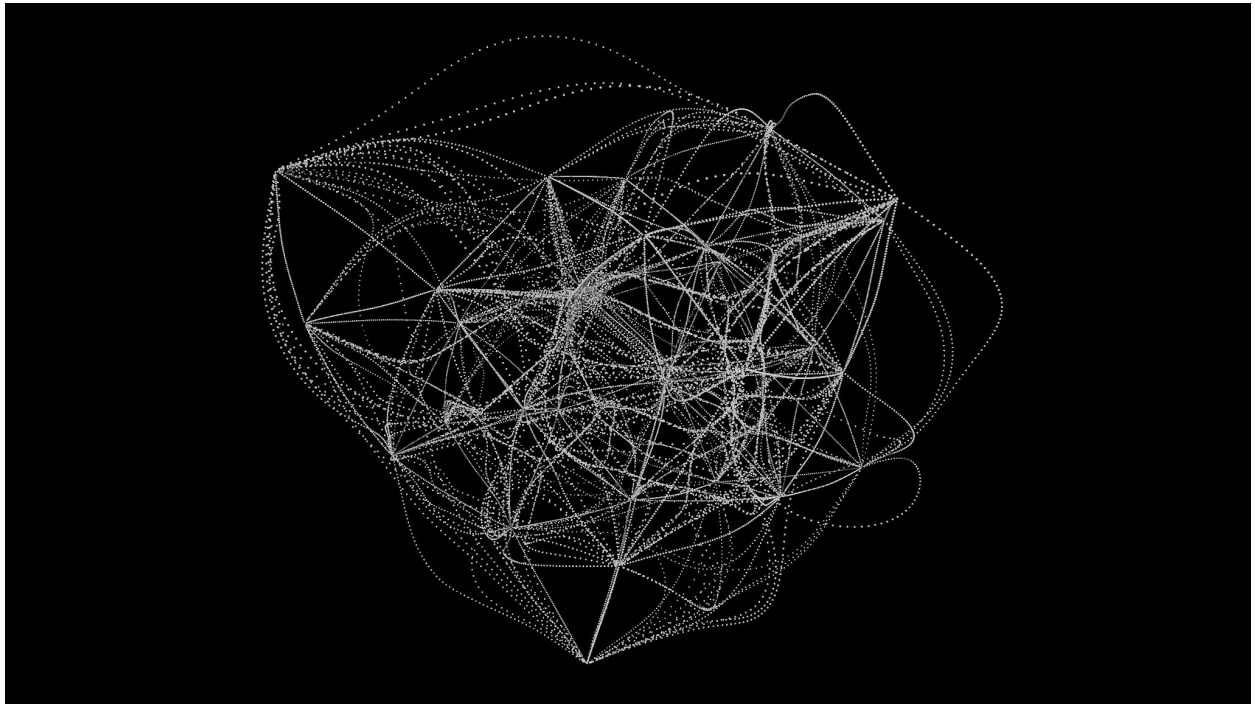
Variational calculation result: cavity connectivity in L-J fluid

The largest cavities in a simulated sample Lennard-Jones fluid were exhaustively paired to form variational paths.

These paths were then iterated to minimize energy at all intermediate points.

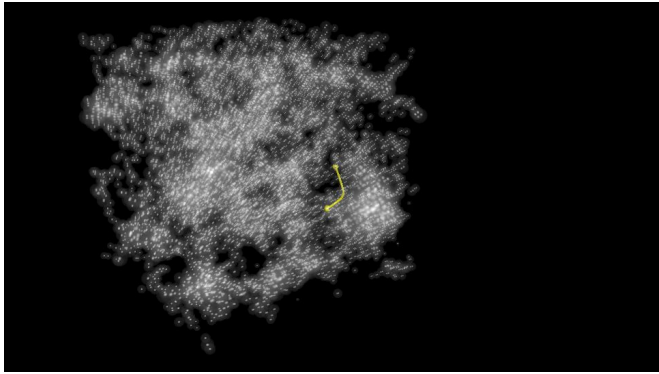
The fluid and subsequent variational calculation are performed with periodic boundaries. Only the pairings within a single period are represented.

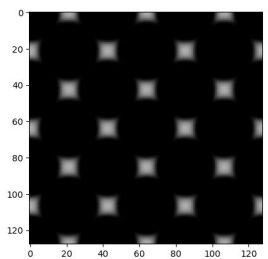
Even for a small sample, the calculation presents unique challenges in generating a satisfying visual representation of the result.

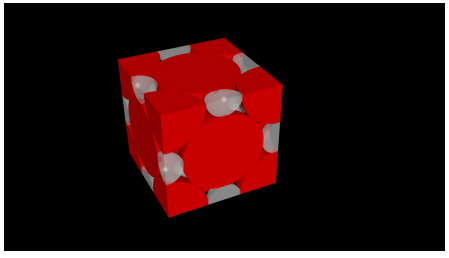


VACUUMMS Show and Tell:

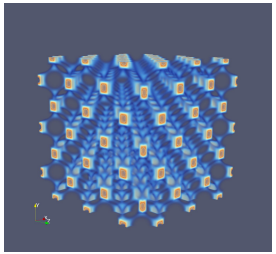
PMP

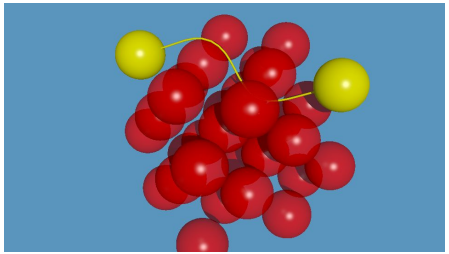




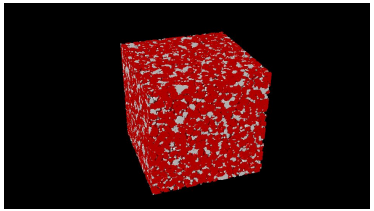


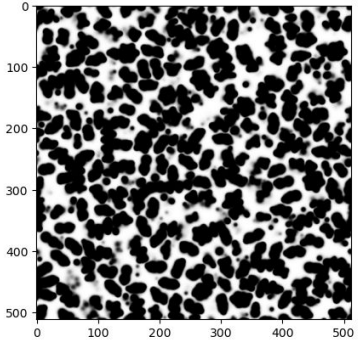
FCC

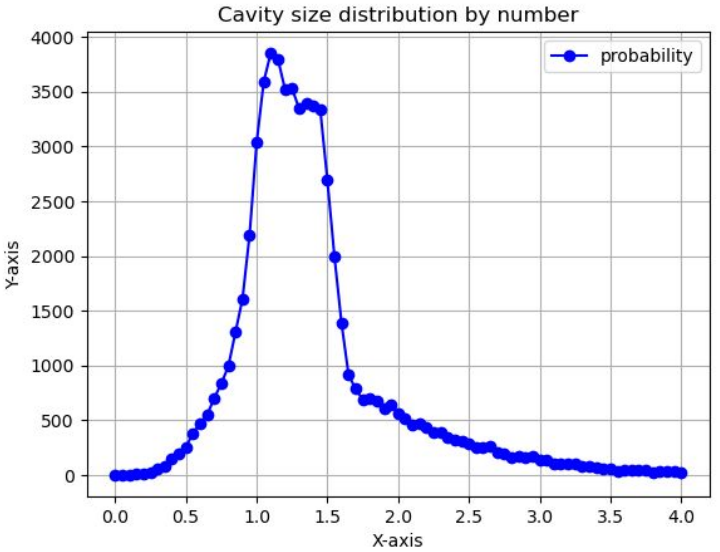


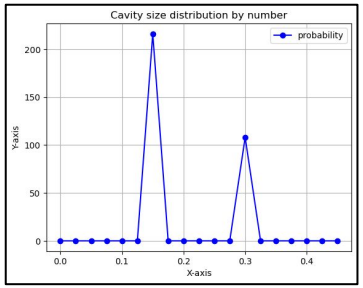


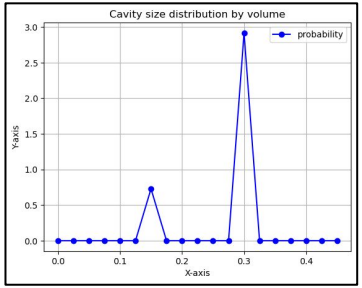
PS











FCC Variational Example

Showcases the variational path calculation between atom positions in an FCC unit cell.

[1]:
```
import vacuumms as v
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
from IPython.display import Image
```

Load configuration and replicate image

[2]:
```
gfg=v.Configuration("fcc_unit_cell.gfg")
gfg.setBoxDimensions([1,1,1])
gfg.replicate([2,2,2])   # make replicas 3-deep in each direction
```

Select two points and remove atoms

[ ]:
```
s=13       # index of start point
e=30       # index of end point

# pull these points out of the configuration
start=gfg.recordAt(s)
end=gfg.recordAt(e)
gfg.deleteRecordAt(e)
gfg.deleteRecordAt(s)

# add them to a separate configuration to be used later for rendering
endpoints=v.Configuration()
endpoints.pushBack(start)
endpoints.pushBack(end)
```

Calculate variational path between endpoints

variables: startpoint, endpoint, sigma, epsilon, # of iterations, configuration)
tuning parameters: alpha, alpha_max, delta_max, beta

[ ]:
```
var=v.Variational3D(start.getXYZ(), end.getXYZ(), 1, 1, 25, gfg)
var.setVerbose(0)       # Set a verbosity level for more/less output when tuning/debugging.
var.setDeltaMax(0.025)  # Set maximum step size
var.printValues()       # Show starting values for iteration, evenly spaced by default.
```

Generate a scene object to display the result

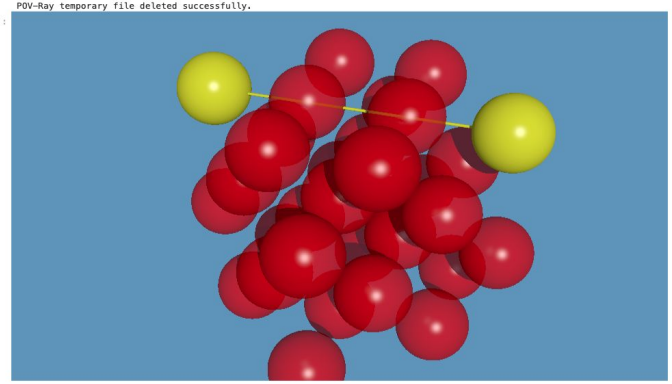
[5]:
```
scene = v.Scene()
vc = v.VariationalComponent(var)
scene.addSceneComponent(vc)

cc = v.ConfigurationComponent(gfg)
cc.setTransmit(0.25)
cc.setPhong(.5)
cc.rescale(0.75) # make the diameter smaller to show trajectory
scene.addSceneComponent(cc)

ep = v.ConfigurationComponent(endpoints)
ep.setColor("Yellow")
ep.setTransmit(0.0)
ep.setPhong(.5)
ep.rescale(0.75)
scene.addSceneComponent(ep)

scene.setCameraLocation([3,4,5])
scene.setCameraLookAt([1,1,1])
scene.applyAmbientLight()
scene.setRenderDimensions(960,540)
scene.clearLightSources()
scene.addLightSource([100,200,300],"White")
scene.setBackgroundColor("rgb <.1, .3, .5>")
```

Render the scene before iterating

[6]:
```
scene.renderScene("var.png")
scene.createSceneFile("x.pov")
Image(filename='var.png')
```

```
POV-Ray render completed successfully.
POV-Ray temporary file deleted successfully.
```

[6]:



Iterate

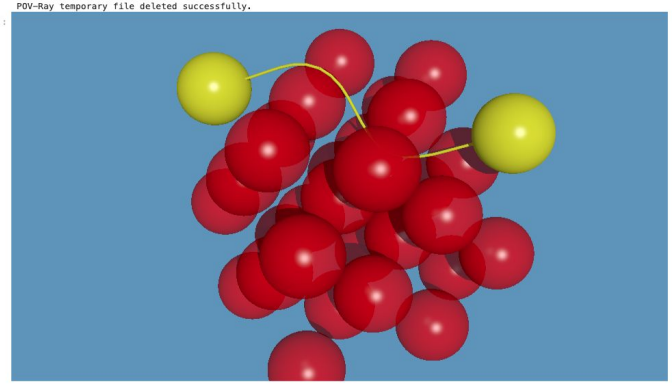
Jitter the initial condition, then perform 50 iterations. The 'shrinkage' here is a reflection of how much the curve expands/contracts with the current iteration.

[ ]:
```
var.jitter(0.01)     # Apply small random noise to nudge points away from saddles.
for _ in range(50):  # Nudge curve 50 times
    shrinkage = var.adaptiveIterateAndUpdate()
    print(_, shrinkage)
```

[8]:
```
scene.renderScene("var.png")
scene.createSceneFile("x.pov")
Image(filename='var.png')
```

```
POV-Ray render completed successfully.
POV-Ray temporary file deleted successfully.
```

[8]:



PMP Example

Loads a PMP configuration from a LAMMPS frame to showcase the cavity sizing operation.

[1]:
```
import vacuumms as v
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
from IPython.display import Image
import numpy as np
```

Generate cavities and view distribution

[7]:
```
gfg=v.LAMMPSConfiguration("PMP.lmps")  # Load configuration from file
ddx = v.DDX(gfg)                       # create the DDX (cavity-search) operation
ddx.setNumberOfSamples(10000)          # Generate 10000 cavity samples
ddx.execute()                          # Execute the cavity sizing operation
cavs=ddx.getResult()                   # Retrieve the cavity result from the ddx operation
cavs.scrubDuplicates()                 # Remove duplicate cavities
csd = v.CavitySizeDistribution(cavs)   # Create the distribution container object
csd.setBinWidth(0.25)                  # Width of bins
csd.setNumberOfBins(25)                # Number of bins for sorting
csd.setStartingValue(0.0)              # Starting bin size
csd.generate()                         # Generate the distribution
csd.applyWeightExponent(3)             # For a volume-weighted distribution
csd.smooth(1)                          # Apply three-point smoothing 1 time
csd                                    # Display the ASCII rendered result
```

[7]:
```
0.000000:
0.250000:
0.500000:       ****
0.750000:       ************************************
1.000000:       ************************************************************************************************
1.250000:       ************************************************************************************************************
1.500000:       ********************************************************************************
1.750000:       ************************************************************************
2.000000:       ************************************************************************************************
2.250000:       ************************************************************************************************
2.500000:       ********************************************************************************
2.750000:       ************************************************************************
3.000000:       ********************************************************
3.250000:       ************************************************
3.500000:       ****************************************
3.750000:       ****************************
4.000000:       ************************
4.250000:       ****************
4.500000:       **********
4.750000:       *******
5.000000:       ******
5.250000:       *****
5.500000:       **
5.750000:
6.000000:
```

Select two large cavities to generate a variati

[9]:
```
start = cavs.recordAt(131).getXYZ()
end = cavs.recordAt(254).getXYZ()

var=v.Variational3D(start, end, 1, 1, 100, g
var.setDeltaMax(0.025)
var.setAlpha(1.0)                      # Starting value of ADAM parameter for gradient descent
var.setAlphaMax(10)                    # Maximum allowed value
var.setVerbose(0)                      # Set verbose level to zero to limit output, higher for debugging and tuning
```

Iterate over the trajectory

[ ]:
```
for _ in range(1000):                                  # Perform 1000 iterations
    shrinkage = var.adaptiveIterateAndUpdate()         # Iterate and update the curve
```

Generate scene object to render result

[7]:
```
scene = v.Scene()                      # Scene object

vc = v.VariationalComponent(var)       # Variational curve scene component
vc.setDiameter(0.1)                    # Diameter of trajectory when displayed
scene.addSceneComponent(vc)            # Add to the scene

cav2=v.CavityConfiguration()           # Create a configuration for just the endpoint cavities
cav2.pushBack(cavs.recordAt(131))
cav2.pushBack(cavs.recordAt(254))
cav_scene2=v.CavityComponent(cav2)     # Create a scene component for the endpoint cavities
cav_scene2.setPhong(0.7)               # make it shiny
cav_scene2.setTransmit(0.0)            # make it opaque
cav_scene2.setColor("Yellow")          # make it Yellow
scene.addSceneComponent(cav_scene2)    # Add it to the scene

cav_scene=v.CavityComponent(cavs)      # Create a scene component for rest of the cavities
cav_scene.setPhong(0.3)                # make it shiny
cav_scene.setTransmit(0.99)            # make it nearly transparent
cav_scene.setColor("White")            # make it White
scene.addSceneComponent(cav_scene)     # Add it to the scene
```

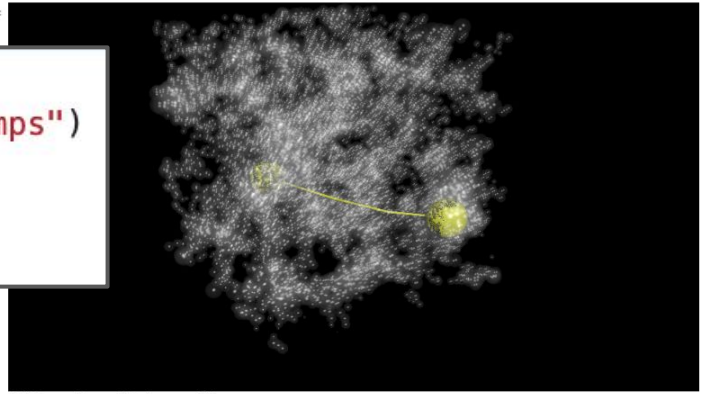
Tune the rendering parameters

[9]:
```
scene.setCameraLocation([44,33,66])
_end = np.array(end, dtype=np.float32)
_start = np.array(start, dtype=np.float32)
scene.setCameraLookAt((_end + _start)/2)
scene.setRenderDimensions(960,540)
scene.clearLightSources()
scene.addLightSource([1000,2000,3000],"White")
scene.applyAmbientLight()

# Scale the displayed ojbects for better visibility
cav_scene.rescale(0.7)
cav_scene2.rescale(0.7)
```

Render the scene

[10]:
```
scene.renderScene("PMP-V.png")
Image(filename='PMP-V.png')
```

[10]:



```
POV-Ray render completed successfully.
POV-Ray temporary file deleted successfully.
```


```
import vacuumms as v
gfg = v.LAMMPSConfiguration("PMP.lmps")
ddx = v.DDX(gfg)
ddx.setNumberOfSamples(10000)
ddx.execute()
```

Acknowledgements and Q/A

Thanks to:

- Janani Sampath at the University of Florida for her interest and to her student Mohammed al Otmi for generating molecular configurations for analysis.
- University of Southern California Center for Advanced Research Computing for their interest and support.
- Boise State University for recognizing the value of the work.
- Isaac Sanchez and the UT McKetta Department of chemical engineering for helping originate the effort.
- The University of Texas at Austin and the Texas Advanced Computing Center for material support.
- NIST / NRC postdoctoral fellowship for sponsorship when much of VACUUMMS was built.

Questions and Interest?

https://github.com/VACUUMMS/VACUUMMS

https://vacuumms.org

frankwillmore@gmail.com